

**Міністерство освіти і науки України
Національний технічний університет України “Київський
політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій**

**Лабораторна робота №8
ШАБЛони «COMPOSITE», «FLYWEIGHT»,
«INTERPRETER», «VISITOR»
Варіант: 25. Installer generator**

Виконав:

Студент групи ІА-22

Птачик Р.С.

Перевірив:

Мягкий М.Ю.

Київ 2024

Зміст:

Тема:.....	3
Короткі теоретичні відомості.....	3
Реалізувати не менше 3-х класів відповідно до обраної теми	8
Реалізувати один з розглянутих шаблонів за обраною темою	9
Діаграма класів для шаблону Interpreter.....	12
Проблема, яку допоміг вирішити шаблон Interpreter	12
Переваги використання шаблону Interpreter	13
Код та висновок	14

Тема: шаблони «composite», «flyweight», «interpreter», «visitor».

Мета: ознайомитися з шаблонами «composite», «flyweight», «interpreter», «visitor». Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи:

Тема:

25. Installer generator (iterator, builder, factory method, bridge, interpreter, client-server)

Генератор інсталяційних пакетів повинен мати якийсь спосіб налаштування файлів, що входять в установку, установки вікон з інтерактивними можливостями (галочка - створити ярлик на робочому столі; ввести в текстове поле деякі дані, наприклад, ліцензійний ключ і т.д.). Генератор повинен вивести один файл .exe або .msi.

Короткі теоретичні відомості

Для розробки масштабних корпоративних додатків, що мають велику кількість користувачів (понад 100) та взаємопов'язаних програм, важливо застосовувати сучасні шаблони. Вони дозволяють ефективно організувати доступ до даних, оскільки обсяг даних, кількість клієнтів і складність додатків постійно зростають.

Шаблон «Active Record» (Активний запис)

Призначення:

Об'єднує в одному об'єкті дані та поведінку, що відповідає одному рядку в базі даних. Цей шаблон простий у реалізації і широко використовується, наприклад, у Ruby on Rails.

Проблема:

При збільшенні кількості запитів складність класу росте, що ускладнює підтримку і розширення.

Рішення:

Частину логіки, пов'язаної із запитом, можна винести в окремий об'єкт, щоб розвантажити основний клас.

Особливості:

- Простота реалізації.
- Вимагає рефакторингу при масштабуванні.

Шаблон «Table Data Gateway» (Шлюз до даних)

Призначення:

Цей шаблон використовує окремий клас для взаємодії з базою даних для кожного класу даних. Усі операції із запитами зосереджені в цьому класі.

Переваги:

- Гнучкість і тестованість.
- Взаємодія з базою даних відокремлюється від даних.

Рішення:

Щоб зменшити дублювання, код взаємодії з базою даних часто виноситься в базовий клас.

Приклади використання:

Ідеально підходить для додатків, де потрібно чітко розмежування між логікою доступу до даних і обробкою даних.

Шаблон «Data Mapping» (Відображення даних)

Призначення:

Цей шаблон вирішує проблему перетворення об'єктів у формат, що приймається базою даних або мережею, і навпаки.

Особливості:

- Виправляє невідповідності типів між об'єктами та джерелом даних.
- Забезпечує чіткий зв'язок між полями таблиці і властивостями об'єкта.

Рішення:

Використання маппера дозволяє розподілити роботу між об'єктами для перетворення даних, зберігаючи відповідність структури БД.

Приклади використання:

Шаблон популярний у програмах із складною структурою даних, де потрібна регулярна синхронізація між базою даних і програмним об'єктом.

Шаблон «Composite»

Призначення:

Шаблон дозволяє об'єднувати об'єкти в деревоподібну структуру для

представлення ієрархій типу «частина-ціле» та обробляти їх уніфіковано. Це спрощує роботу з ієрархічними даними та забезпечує гнучкість.

Використання:

- Підходить, якщо модель програми можна подати у вигляді дерева.
- Наприклад, продукт і коробка мають спільний інтерфейс із методом для отримання вартості.
- Коробка підсумовує ціни вкладених продуктів і коробок.

Переваги:

- Полегшує роботу з деревоподібними структурами.
- Сприяє легкому додаванню нових компонентів.

Недоліки:

- Може призводити до занадто загального дизайну класів.

Шаблон «Flyweight»

Призначення:

Оптимізує використання оперативної пам'яті в програмах із великою кількістю схожих об'єктів шляхом розподілу загальних (внутрішніх) даних між цими об'єктами.

Рішення:

Розділити дані на:

1. **Внутрішні (загальні):** Спільні для всіх об'єктів, наприклад, спрайт чи колір.
2. **Зовнішні (унікальні):** Інформація, специфічна для кожного екземпляра, як-от координати або швидкість.

Приклад:

1. У грі всі частинки мають спільні властивості (спрайт, колір) у класі `ParticleType`, тоді як унікальні властивості (позиція, напрямок) зберігаються у контексті.
2. У базі даних ветеринарної клініки:
 - Загальні дані (порода, колір) — у класі `CatBreed`.
 - Унікальні дані (ім'я, вік, власник) — у класі `Cat`.

Переваги:

- Економить пам'ять завдяки розподілу даних.

Недоліки:

- Ускладнює код через необхідність додаткових класів.
- Вимагає більше обчислювальних ресурсів для обробки зовнішніх даних.

Шаблон «Interpreter»

Призначення:

Дозволяє описувати граматику певної мови та створювати інтерпретатор для її виразів. Граматика описується термінальними та нетермінальними символами, які інтерпретуються в заданому контексті.

Рішення:

Шаблон базується на побудові абстрактного синтаксичного дерева, де:

- **Термінальні вирази** описують кінцеві елементи граматики.
- **Нетермінальні вирази** представляють правила граматики, що об'єднують інші вирази.
- **Контекст** забезпечує глобальну інформацію для інтерпретації.
- **Клієнт** будує дерево і ініціює його обчислення.

Розширення:

Додаючи нові термінальні або нетермінальні символи, можна легко змінювати та розширювати граматику. Крім того, шаблон дозволяє реалізовувати кілька способів інтерпретації для одного дерева.

Приклад:

1. У задачах автоматичного розбору завдань система може перетворювати складний вираз на прості інструкції.
2. У пошуку за шаблоном інтерпретатор може розбивати текст на блоки для перевірки відповідності заданій граматиці.

Структура:

1. **AbstractExpression:** Оголошує метод Інтерпретувати(Context).
2. **TerminalExpression:** Реалізує інтерпретацію термінального символу.
3. **NonterminalExpression:** Описує правила, що включають інші вирази.
4. **Context:** Зберігає глобальні дані для інтерпретації.
5. **Client:** Будує дерево і викликає операцію інтерпретації.

Переваги:

- Простота розширення граматики.
- Легке додавання нових способів обчислення.

Недоліки:

- Складно підтримувати велику кількість правил.
- Може збільшити кількість необхідних класів.

Шаблон «Visitor»**Призначення:**

Дозволяє додавати нові операції до об'єктів ієрархії без зміни їхньої структури. Використовується для групування операцій, що застосовуються до різнотипних об'єктів.

Рішення:

Виділення нової поведінки у клас відвідувача, який виконує дії залежно від типу об'єкта. Об'єкти передають себе у методи відвідувача.

Приклад:

У компіляторі різні об'єкти (виклики методів, умовні оператори) обробляються відвідувачами:

- Для перевірки типів.
- Для генерації коду.

Переваги:

- Легкість додавання нових операцій.
- Зручність групування поведінки.

Недоліки:

- Додавання нових типів об'єктів вимагає змін у всіх відвідувачах.

Реалізувати не менше 3-х класів відповідно до обраної теми

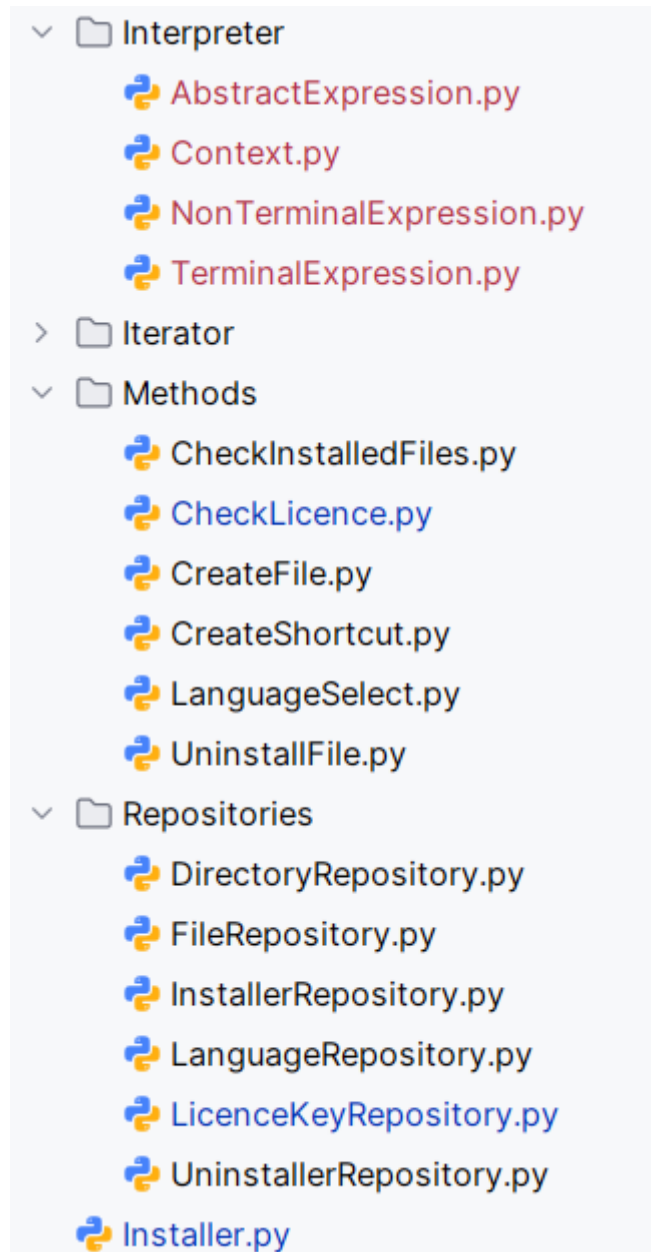


Рис. 1 – Структура проекту

В цій лабораторній роботі було реалізовано частину генератора інсталяційних пакетів з використанням шаблону interpreter, а саме перевірка ліцензійного ключа.

Реалізувати один з розглянутих шаблонів за обраною темою
Згідно з завданням, було використано та реалізовано шаблон interpreter.

1. Context:

```
class Context:  # FryMondo
    def __init__(self, license_key_repository):  # FryMondo
        self.licence_key_repository = license_key_repository
        self.user_input_key = None
```

Рис. 2 – Клас Context

- Контекст, що містить глобальну інформацію для інтерпретації виразів.
- Використовується для передачі введеного користувачем ліцензійного ключа та репозиторію LicenceKeyRepository.
- Забезпечує єдиний доступ до даних, необхідних для інтерпретації, спрощуючи взаємодію між виразами.

2. AbstractExpression:

```
class AbstractExpression: 4 usages  # FryMondo
    def interpret(self, context: Context):  # FryMondo
        raise NotImplementedError("Subclasses must implement 'interpret' method")
```

Рис. 3 – Клас AbstractExpression

- Абстрактний клас, що визначає інтерфейс для всіх виразів.
- Оголошує метод interpret(context), який повинен бути реалізований у підкласах.

3. TerminalExpression:

```
class TerminalExpression(AbstractExpression): 2 usages  #
    def __init__(self, valid_key):  # FryMondo
        self.valid_key = valid_key

    def interpret(self, context: Context):  # FryMondo
        return context.user_input_key == self.valid_key
```

Рис. 4 – Клас TerminalExpression

- Конкретний вираз, що представляє термінальний символ у граматиці.
- Перевіряє, чи відповідає введений користувачем ліцензійний ключ (context.user_input_key) дійсному ключу (valid_key).

- Метод `interpret(context)` повертає `True` або `False` залежно від відповідності ключа.

4. NonTerminalExpression:

```
class NonTerminalExpression(AbstractExpression):
    def __init__(self, expressions):  # FryMondo
        self.expressions = expressions

    def interpret(self, context: Context):  # FryMc
        for expression in self.expressions:
            if not expression.interpret(context):
                return False
        return True
```

Рис. 5 – Клас `NonTerminalExpression`

- Конкретний вираз, що об'єднує кілька інших виразів (термінальних або нетермінальних).
- Метод `interpret(context)` перевіряє всі вкладені вирази, повертаючи `True`, якщо всі вони виконуються, і `False`, якщо хоча б один не виконується.
- Забезпечує можливість створення складних граматичних правил.

5. LicenceKeyRepository:

```
class LicenseKeyRepository:  # usag
    def __init__(self):  # FryMondo
        self.id = None
        self.licence_key = "1111"

    def get(self):  # FryMondo
        return self.licence_key
```

Рис. 6 – Клас `LicenseKeyRepository`

- Виконує роль джерела даних, надаючи ліцензійний ключ для перевірки.
- Метод `get()` повертає поточний ліцензійний ключ для порівняння.

6. CheckLicense (Client):

```
class CheckLicense: 2 usages  ± FryMondo
    def __init__(self, repository):  ± FryMondo
        self.context = Context(repository)

    def checkLicense(self): 1 usage  ± FryMondo
        terminal_expression = TerminalExpression(self.context.licence_key_repository.get())
        while True:
            self.context.user_input_key = input(get_language().get_message("enter_license_key"))
            if terminal_expression.interpret(self.context):
                print(get_language().get_message("license_key_valid"))
                break
            else:
                print(get_language().get_message("license_key_invalid"))
```

Рис. 7 – Клас CheckLicense

- Відповідає за використання інтерпретатора для перевірки ліцензійного ключа, введеного користувачем.
- Використовує контекст Context і вираз TerminalExpression для перевірки відповідності ключа.
- У разі успішної перевірки повідомляє користувача, що ключ дійсний; в іншому випадку запитує ключ повторно.

7. Installer:

```
class Installer: 1 usage  ± FryMondo
    def __init__(self, installer_repository):  ± FryMondo
        self._installer_repository = installer_repository

    def check_license(self): 1 usage  ± FryMondo
        client = CheckLicense(self._installer_repository.get_license_key_repo())
        client.checkLicense()
```

Рис. 8 – Клас Installer

- Інтегрує перевірку ліцензійного ключа в загальну логіку інсталятора.
- Метод check_license() створює об'єкт CheckLicense і викликає метод для перевірки ключа.

Результат:

```
Selected language: Українська
Введіть ліцензійний ключ: 1234
Невірний ліцензійний ключ. Спробуйте ще раз.
Введіть ліцензійний ключ: qwerty
Невірний ліцензійний ключ. Спробуйте ще раз.
Введіть ліцензійний ключ: 1111
Ліцензійний ключ вірний. Продовження інсталяції...
```

Рис. 9 – Результат перевірки ліцензійного ключа

Діаграма класів для шаблону Interpreter

Діаграма класів для шаблону Interpreter:

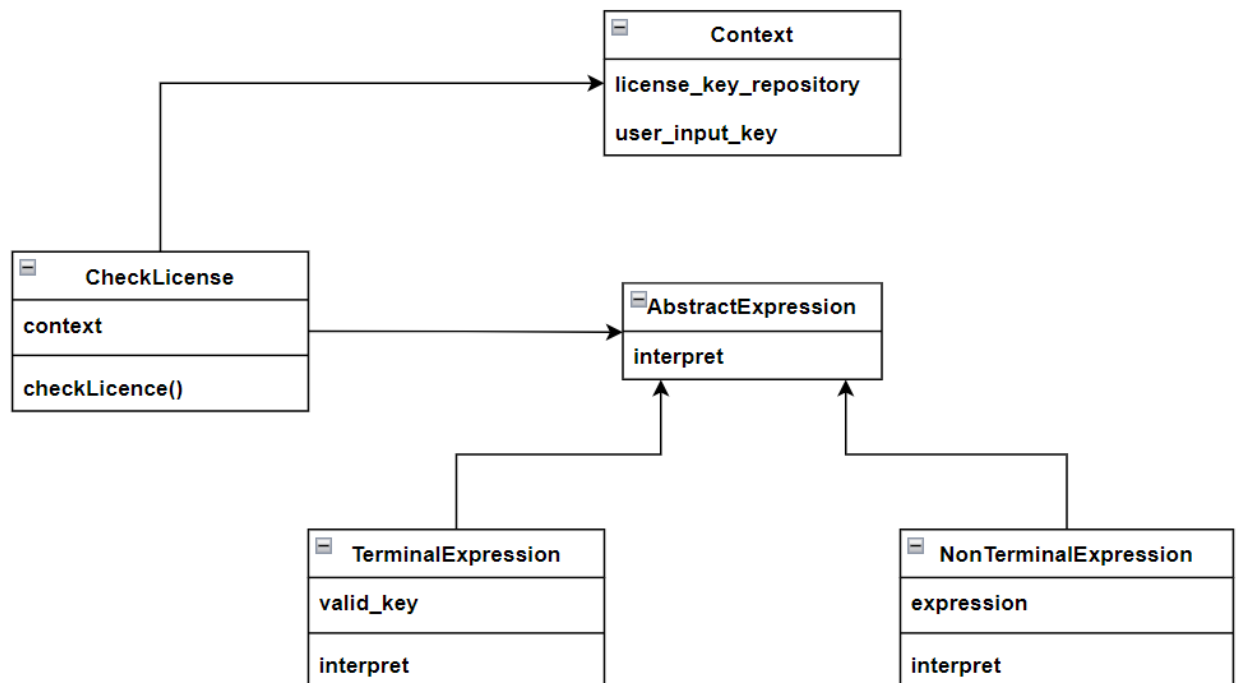


Рис. 10 – Діаграма класів для шаблону interpreter

Проблема, яку допоміг вирішити шаблон Interpreter

Використання шаблону Interpreter у проекті "Installer Generator" вирішило проблему реалізації гнучкого та масштабованого механізму перевірки ліцензійного ключа на основі заданих правил. Завдяки цьому шаблону система стала здатною обробляти як прості, так і складні логічні правила без порушення основного коду.

Завдяки шаблону Interpreter вдалося:

- Розділити логіку перевірки ключів на окремі вирази: конкретні перевірки реалізовані у вигляді термінальних (TerminalExpression) та нетермінальних (NonTerminalExpression) виразів, що полегшило управління логікою.
- Забезпечити масштабованість: нові правила для перевірки ліцензійного ключа можна легко додати шляхом створення нових виразів без модифікації існуючого коду.
- Інкапсулювати правила перевірки: уся логіка перевірки зосереджена в окремих класах, що підвищує читабельність та спрощує тестування.
- Забезпечити модульність: логіка перевірки ключів ізольована від інших компонентів системи, що дозволяє її незалежно змінювати чи розширювати.

Це рішення зробило перевірку ліцензійного ключа у проекті "Installer Generator" структурованою, розширюваною та легкою для підтримки. Шаблон Interpreter дозволив побудувати гнучкий механізм роботи з правилами, які можна адаптувати до нових вимог без порушення існуючої архітектури.

Переваги використання шаблону Interpreter

Модульність:

- Кожне правило перевірки реалізовано у вигляді окремого класу (TerminalExpression, NonTerminalExpression), що ізолює логіку та полегшує її тестування й розширення.
- Вся логіка перевірки винесена з основного коду інсталятора, що підвищує його читабельність.

Масштабованість:

- Додавання нових правил перевірки не потребує змін у вже існуючому коді. Наприклад, можна додати новий вираз, який перевіряє формат ліцензійного ключа, лише створивши новий клас.

Гнучкість:

- Комбінація термінальних і нетермінальних виразів дозволяє створювати прості та складні граматичні конструкції.

- Наприклад, клас NonTerminalExpression може перевіряти послідовність кількох правил для ключа, об'єднуючи інші вирази в єдине правило.

Інкапсуляція логіки:

- Усі правила перевірки зосереджені в класах, що реалізують AbstractExpression. Це спрощує підтримку й унеможливорює дублювання логіки.

Читабельність і підтримка:

- Логіка перевірки стає зрозумілою та легко розширюваною, оскільки правила представлені в окремих класах з чіткою відповідальністю.

Шаблон Interpreter забезпечив у проекті "Installer Generator" гнучкість і структурованість у реалізації логіки перевірки ліцензійного ключа. Його застосування дозволило ефективно працювати з простими та складними правилами, використовуючи модульний підхід до їхньої реалізації.

Код та висновок

Код можна знайти за посиланням:

<https://github.com/FryMondo/TRPZ/tree/master/InstallGenerator>

Висновок: У цій лабораторній роботі реалізовано шаблон Interpreter для перевірки ліцензійного ключа. Це рішення забезпечило гнучкість, масштабованість та модульність системи, що дозволило легко додавати нові правила перевірки без змін основного коду. Застосування шаблону значно спростило структуру, підвищило читабельність та спростило підтримку логіки перевірки ключів у програмі.